

DSA0602-Data Handling and Visualization

Lab Experiments 21-25

S.

NAME: R. Reddy Balaji

Reg No 192424122

SET 21 – ENERGY CONSUMPTION ANALYSIS

Experiment 1(a): Histogram and Density Plot for Units Consumed

Aim

To import the energy consumption dataset in R and plot a histogram and density plot for Units_Consumed to analyze the energy consumption pattern.

Procedure

1. Import Energy_Consumption_Data.csv into R.
2. Display the dataset and check its structure.
3. Select the Units_Consumed variable.
4. Create a histogram to show the frequency distribution.
5. Add a density curve to identify the overall distribution pattern.
6. Observe and compare the consumption values.

R Code

```
# Import dataset energy <-  
read.csv("Energy_Consumption_Data.csv")  
  
# Display dataset print(energy)  
  
# Check structure  
str(energy)
```

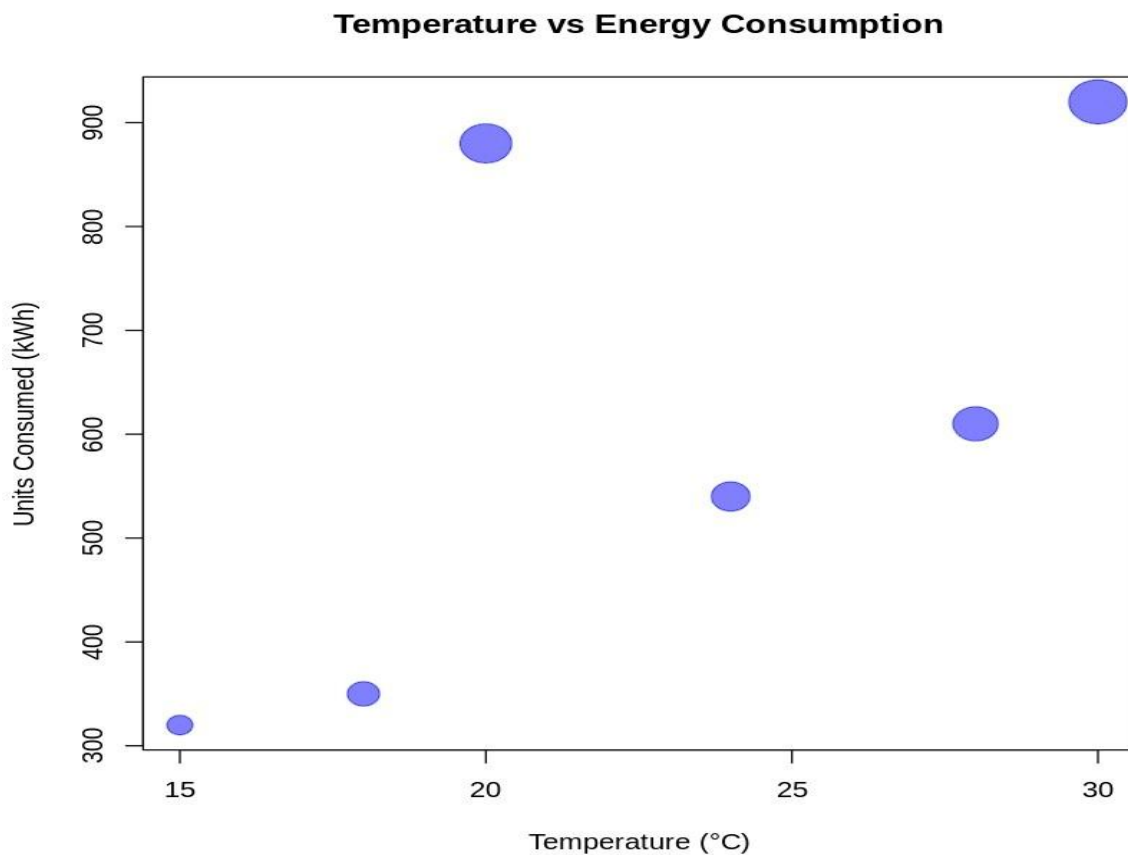
```
# Histogram hist(energy$Units_Consumed,
probability = TRUE, breaks = 6, col
= "lightblue", main = "Histogram of
Units Consumed", xlab = "Units
Consumed (kWh)", ylab = "Density")

# Density curve lines(density(energy$Units_Consumed),
col = "red",
lwd = 2)
```

Result / Output

The histogram and density plot successfully display the distribution of energy consumption. The consumption values range from **320 kWh to 920 kWh**. Residential users have lower consumption, while Industrial users have considerably higher consumption.

Output: A histogram with a red density curve showing the distribution of Units_Consumed.



Experiment 1(b): Temperature vs Units Consumed Bubble Scatter Plot

Aim

To create a scatter plot between Temperature and Units_Consumed, represent Peak_Hours using bubble size, and apply transparency to analyze the relationship between temperature and energy consumption.

Procedure

1. Import the energy consumption dataset.
2. Select Temperature and Units_Consumed.
3. Create a scatter plot.
4. Represent Peak_Hours using bubble size.
5. Apply transparency to the bubbles.
6. Add a regression line to identify the trend.
7. Observe the relationship between temperature and energy consumption.

R Code

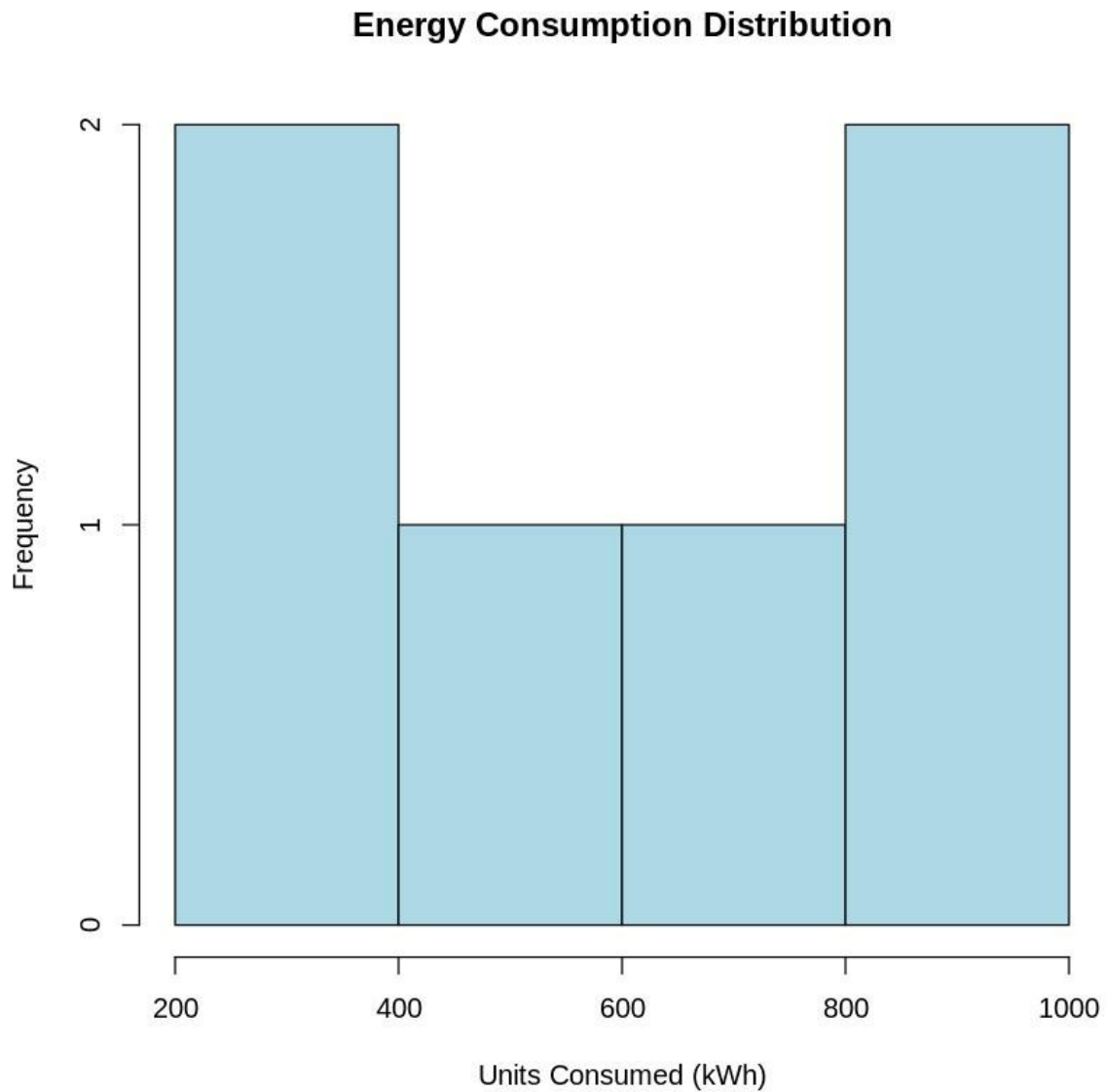
```
# Bubble scatter plot plot(energy$Temperature,
energy$Units_Consumed,
  pch = 21,    cex = energy$Peak_Hours /
4,    bg = rgb(0.2, 0.5, 0.8, 0.45),    col =
"darkblue",    main = "Temperature vs Units
Consumed",    xlab = "Temperature (°C)",
ylab = "Units Consumed (kWh)")

# Regression line abline(lm(Units_Consumed
~ Temperature,
  data = energy),
col = "red",    lwd
= 2)
```

Result / Output

The scatter plot shows a **positive relationship** between temperature and energy consumption. As temperature increases, energy consumption generally increases. Larger bubbles indicate higher Peak_Hours.

Output: A transparent bubble scatter plot with a regression line showing the relationship between Temperature and Units Consumed.



Experiment 1(c): Average Renewable Usage by Sector

Aim

To calculate the average Renewable_Usage for each sector and represent renewable energy adoption using a bar chart.

Procedure

1. Import the energy consumption dataset.
2. Group the data according to Sector.
3. Calculate the average Renewable_Usage for each sector.
4. Create a bar chart for the calculated averages.
5. Add value labels to the bars.
6. Compare renewable energy adoption among the sectors.

R Code

```
# Calculate average Renewable Usage by Sector avg_renew
```

```
<- aggregate(Renewable_Usage ~ Sector,  
             data = energy,  
             FUN = mean)
```

```
# Display average values print(avg_renew)
```

```
# Create bar chart
```

```
barplot(avg_renew$Renewable_Usage,  
names.arg = avg_renew$Sector, col =  
"seagreen", main = "Average Renewable  
Usage by Sector", xlab = "Sector", ylab =  
"Average Renewable Usage (%)", ylim = c(0,  
30))
```

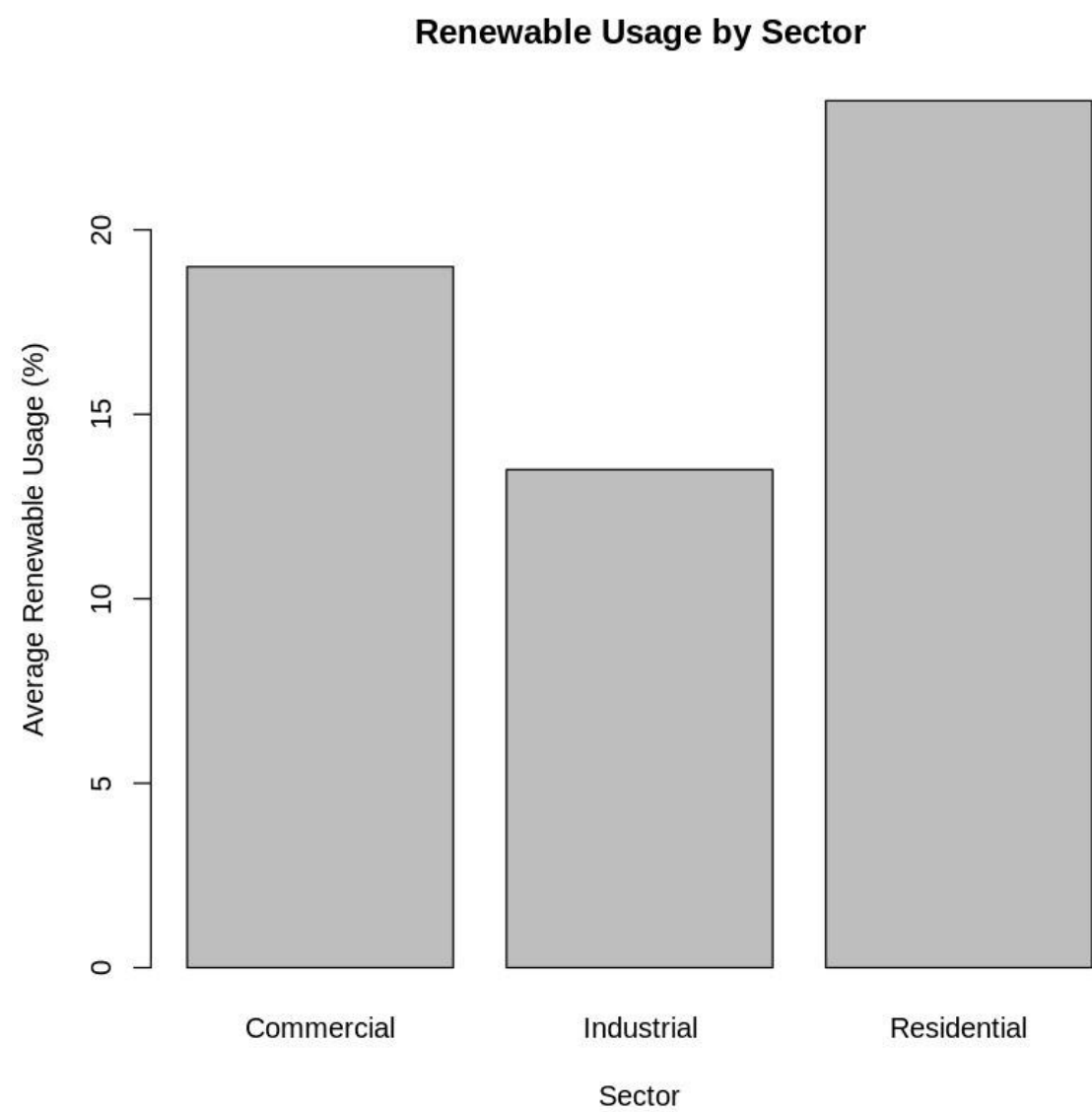
```
# Add value labels
```

```
text(seq_along(avg_renew$Renewable_Usage),  
avg_renew$Renewable_Usage, labels =  
round(avg_renew$Renewable_Usage, 1),
```

pos = 3)

Result / Output

The average renewable energy usage is:



Sector	Average Renewable Usage
--------	-------------------------

Residential	23.5%
-------------	-------

Commercial	19.0%
------------	-------

Industrial	13.5%
------------	-------

The **Residential** sector has the highest renewable energy adoption, while the **Industrial** sector has the lowest.

Output: A bar chart displaying the average renewable energy usage for Residential, Commercial and Industrial sectors.

Experiment 1(d): Tableau Energy Consumption Dashboard

Aim

To create an interactive Tableau dashboard showing monthly energy usage trends and a Sector vs Region heatmap with filters for Month and Sector.

Procedure

1. Open Tableau Desktop.
2. Select **Connect** → **Text File**.
3. Import Energy_Consumption_Data.csv.
4. Create a line chart by placing Month on Columns and Units_Consumed on Rows.
5. Create a heatmap using Sector, Region, and Units_Consumed.
6. Add filters for Month and Sector.
7. Combine the charts into a dashboard.
8. Add titles, legends and interactive tooltips.

R Code

No R code is required for this question because the visualization is created in Tableau.

Tableau Steps

Line Chart:

Columns → Month

Rows → SUM(Units_Consumed)

Marks → Line

Title:

Monthly Energy Usage Trend

Heatmap:

Rows → Sector

Columns → Region

Color → SUM(Units_Consumed) Label

→ SUM(Units_Consumed)

Marks → Square

Title:

Energy Consumption Heatmap

Filters:

Month Sector

Add both charts to a **Dashboard** and apply the filters to the required worksheets.

Result / Output

The interactive Tableau dashboard successfully displays the **monthly energy usage trend** and the **Sector vs Region energy consumption heatmap**. The Month and Sector filters allow users to interactively analyze specific energy consumption patterns.

Output: An interactive dashboard containing a line chart, heatmap, Month filter and Sector filter.

SET 22 – TIME SERIES ANALYSIS

Experiment 2(a): Monthly Sales Time Series Line Chart

Aim

To create a time series line chart in R to visualize the trend in monthly sales.

Dataset

Month Sales (\$)

January 15000

February 18000

March 22000

April 20000

May 23000

Procedure

1. Create the monthly sales dataset in R.
2. Convert the sales data into a time series object.
3. Plot the time series using a line chart.
4. Label the X-axis as Month and Y-axis as Sales.
5. Add a suitable title.
6. Observe the sales trend.

R Code

```
# Create sales data sales <- c(15000, 18000,  
22000, 20000, 23000)
```

```
# Create time series
```

```
sales_ts <- ts(sales,  
start = c(2026, 1),  
frequency = 12)
```

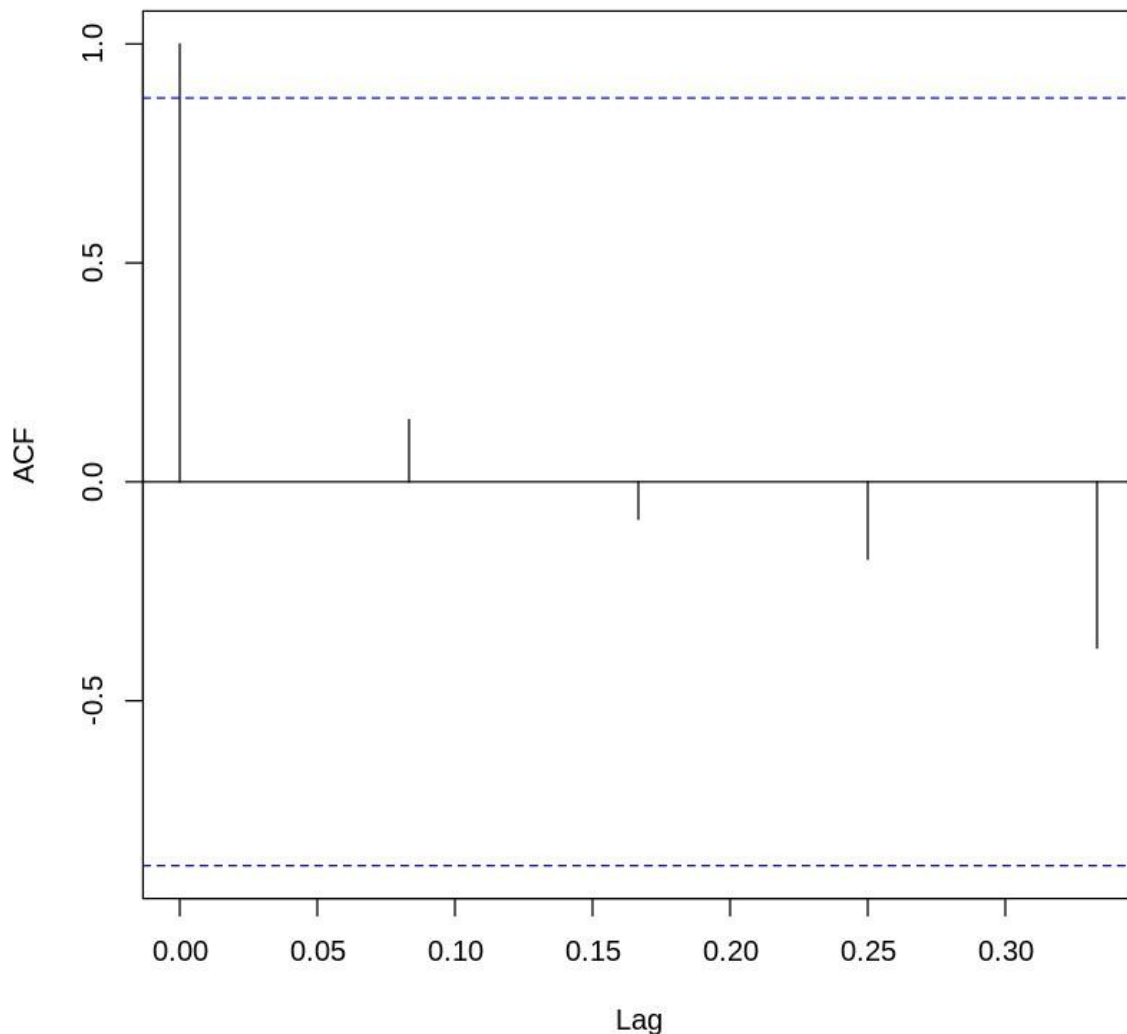
```
# Plot time series plot(sales_ts,  
type = "o",    pch = 16,    col =  
"blue",    main = "Monthly Sales  
Trend",    xlab = "Month",  
ylab = "Sales ($)")
```

Result / Output

The time series line chart shows the monthly sales trend. Sales increase from **\$15,000 in January to \$22,000 in March**, decrease slightly to **\$20,000 in April**, and then increase to **\$23,000 in May**.

Output: A line chart representing the monthly sales trend.

Autocorrelation of Monthly Sales



Experiment 2(b): Advertising Budget vs Monthly Sales

Aim

To analyze the relationship between advertising budget and monthly sales using a scatter plot in R.

Procedure

1. Create a dataset containing monthly sales and advertising budget.
2. Plot Advertising Budget on the X-axis.
3. Plot Sales on the Y-axis.
4. Calculate the correlation between advertising budget and sales.

5. Add a regression trend line.
6. Analyze the relationship.

R Code

```
# Create dataset sales_data
<- data.frame(
  Month = c("January", "February", "March",
            "April", "May"),
  Sales = c(15000, 18000, 22000, 20000, 23000),
  Advertising_Budget = c(5000, 6000, 7500, 7000, 8000)
)

# Scatter plot plot(sales_data$Advertising_Budget,
sales_data$Sales,    pch = 19,    col = "blue",
main = "Advertising Budget vs Monthly Sales",
xlab = "Advertising Budget ($)",    ylab = "Sales
($)")

# Add regression line abline(lm(Sales
~ Advertising_Budget,
    data = sales_data),
col = "red",    lwd =
2)

# Calculate correlation
cor(sales_data$Advertising_Budget,
sales_data$Sales)
```

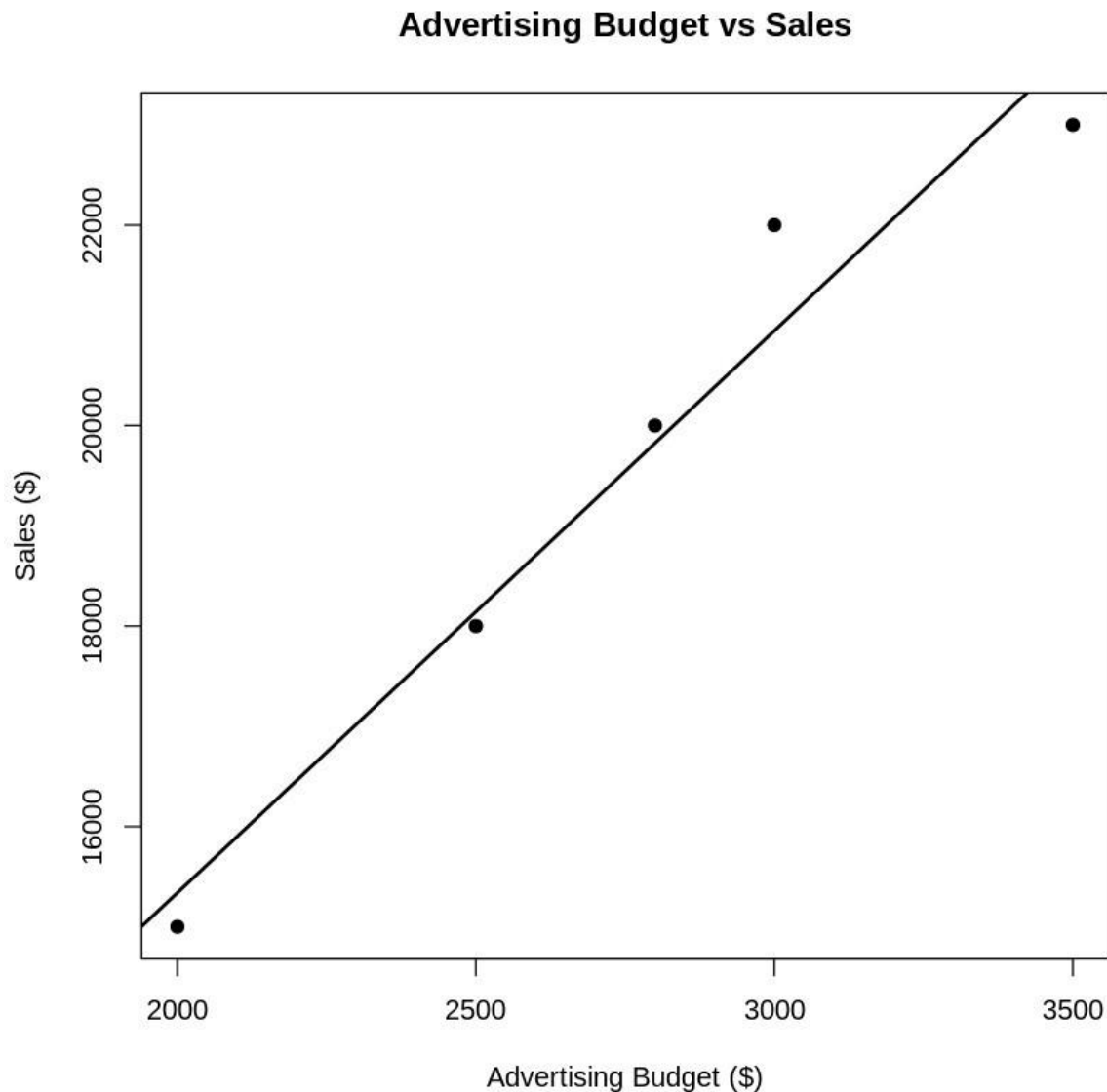
Result / Output

The scatter plot shows a **positive relationship** between advertising budget and monthly sales. As advertising expenditure increases, sales generally increase.

The correlation value is approximately **0.97**, indicating a strong positive relationship in the example data.

Output: A scatter plot with a regression line showing the relationship between advertising budget and sales.

Note: The question does not provide advertising-budget values, so the code uses example values. If your faculty has given actual advertising-budget data, replace those values.



Experiment 2(c): Tableau Sales Trend and Product Distribution Dashboard

Aim

To create an interactive Tableau dashboard combining a line chart for monthly sales trends and a pie chart showing sales distribution across products.

Procedure

1. Open Tableau Desktop.
2. Import the Monthly Sales dataset.
3. Create a line chart for monthly sales.
4. Create a pie chart for product-wise sales distribution.
5. Add labels and legends.
6. Combine both charts into a dashboard.
7. Add interactive filters if required.

R Code

No R code is required for this question because the visualization is created in Tableau.

Tableau Procedure Line

Chart:

Columns → Month

Rows → SUM(Sales)

Marks → Line

Chart title: Monthly

Sales Trend Pie

Chart:

Color → Product

Angle → SUM(Sales)

Label → SUM(Sales)

Chart title:

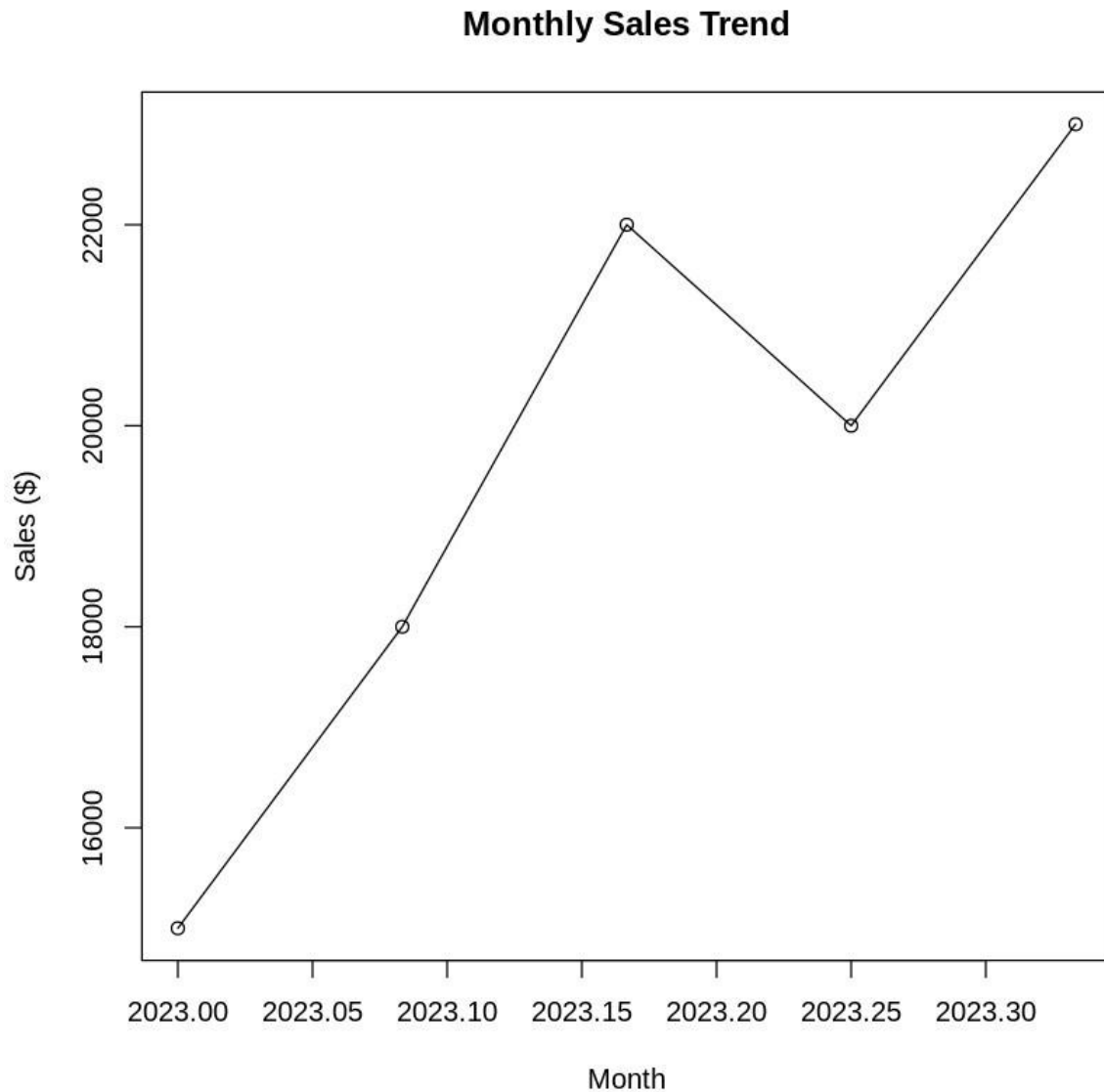
Sales Distribution by Product Then:

1. Select **New Dashboard**.
2. Drag the line chart onto the dashboard.
3. Drag the pie chart onto the dashboard.
4. Add filters where appropriate.
5. Add titles and legends.
6. Save the Tableau workbook.

Result / Output

The Tableau dashboard displays the **monthly sales trend** using a line chart and the **distribution of sales across products** using a pie chart.

Output: An interactive Tableau dashboard containing a sales trend line chart and product distribution pie chart.



Note: The supplied dataset contains only Month and Sales, not Product. Use the Product field from the complete dataset provided by your faculty for the pie chart.

Experiment 2(d): Autocorrelation Plot

Aim

To create an autocorrelation plot in R to identify possible seasonality and relationships between observations at different time lags.

Procedure

1. Create the monthly sales time series.
2. Apply the acf() function.
3. Generate the autocorrelation plot.
4. Observe the correlation at different lags.
5. Determine whether any repeating seasonal pattern is visible.

R Code

```
# Create sales data sales <- c(15000, 18000,  
22000, 20000, 23000)
```

```
# Create time series
```

```
sales_ts <- ts(sales,  
start = c(2026, 1),  
frequency = 12)
```

```
# Autocorrelation plot acf(sales_ts, main =  
"Autocorrelation of Monthly Sales",  
lag.max = 4)
```

Result / Output

The autocorrelation plot shows the relationship between current sales and previous monthly sales values.

Since the dataset contains only **five months of observations**, there are not enough data points to reliably identify seasonal patterns. Therefore, the autocorrelation result should be interpreted cautiously.

Output: An autocorrelation plot showing correlations at different time lags.

SET 23 – EMPLOYEE PERFORMANCE ANALYSIS

Experiment 3(a): Department-wise Employee Distribution

Aim

To create a bar chart in R to visualize the distribution of employees across different departments.

Dataset

Employee ID	Department	Years of Service	Performance Score
1	Sales	5	85
2	HR	3	92
3	Marketing	7	78
4	Sales	4	90
5	HR	2	76

1	Sales	5	85
2	HR	3	92
3	Marketing	7	78
4	Sales	4	90
5	HR	2	76

Procedure

1. Create the employee performance dataset in R.
2. Count the number of employees in each department.
3. Create a bar chart using the department counts.
4. Add a suitable title.
5. Label the X-axis and Y-axis.
6. Add values on top of the bars.
7. Observe the employee distribution.

R Code

```
# Create employee dataset

emp <- data.frame(
  Employee_ID = c(1, 2, 3, 4, 5),
  Department = c("Sales", "HR", "Marketing",
                 "Sales", "HR"),
  Years_of_Service = c(5, 3, 7, 4, 2),
  Performance_Score = c(85, 92, 78, 90, 76)
)
```



```
# Count employees by department dept_count
<- table(emp$Department)

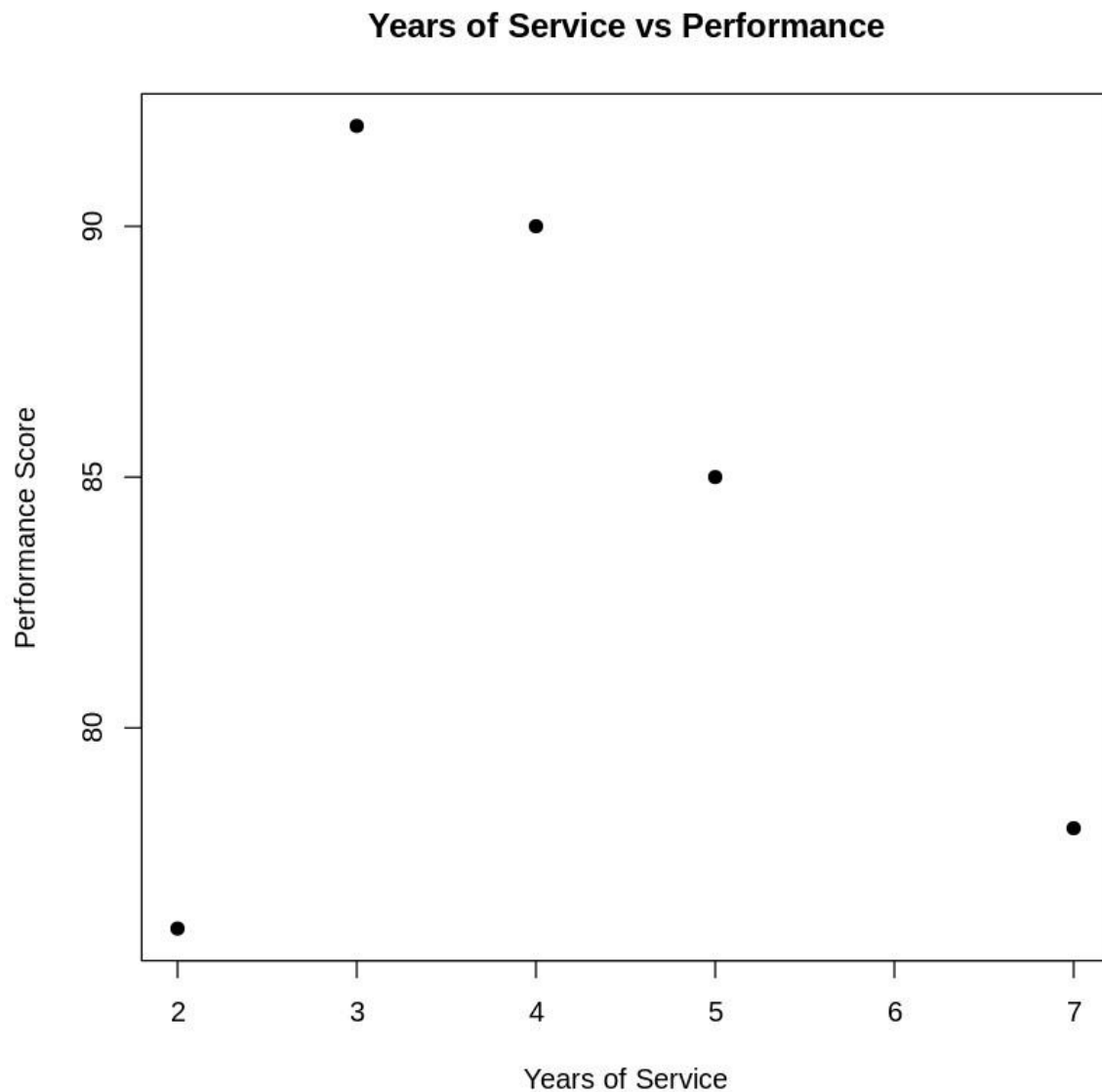
# Create bar chart barplot(dept_count, col =
"skyblue", main = "Employee Distribution by
Department", xlab = "Department", ylab =
"Number of Employees")

# Add values
text(seq_along(dept_count),
dept_count, labels =
dept_count, pos = 3)
```

Result / Output

The bar chart shows that **Sales has 2 employees, HR has 2 employees, and Marketing has 1 employee.**

Output: A bar chart displaying the number of employees in each department.



Experiment 3(b): Employee Performance Trend

Aim

To create a line chart in R to visualize the performance scores of employees according to their years of service.

Procedure

1. Create the employee dataset.
2. Arrange employees according to their years of service.
3. Plot Years of Service on the X-axis.
4. Plot Performance Score on the Y-axis.
5. Connect the observations using a line.
6. Add suitable labels and a title.

7. Analyze the performance trend.

R Code

```
# Sort data according to years of service emp
<- emp[order(emp$Years_of_Service), ]

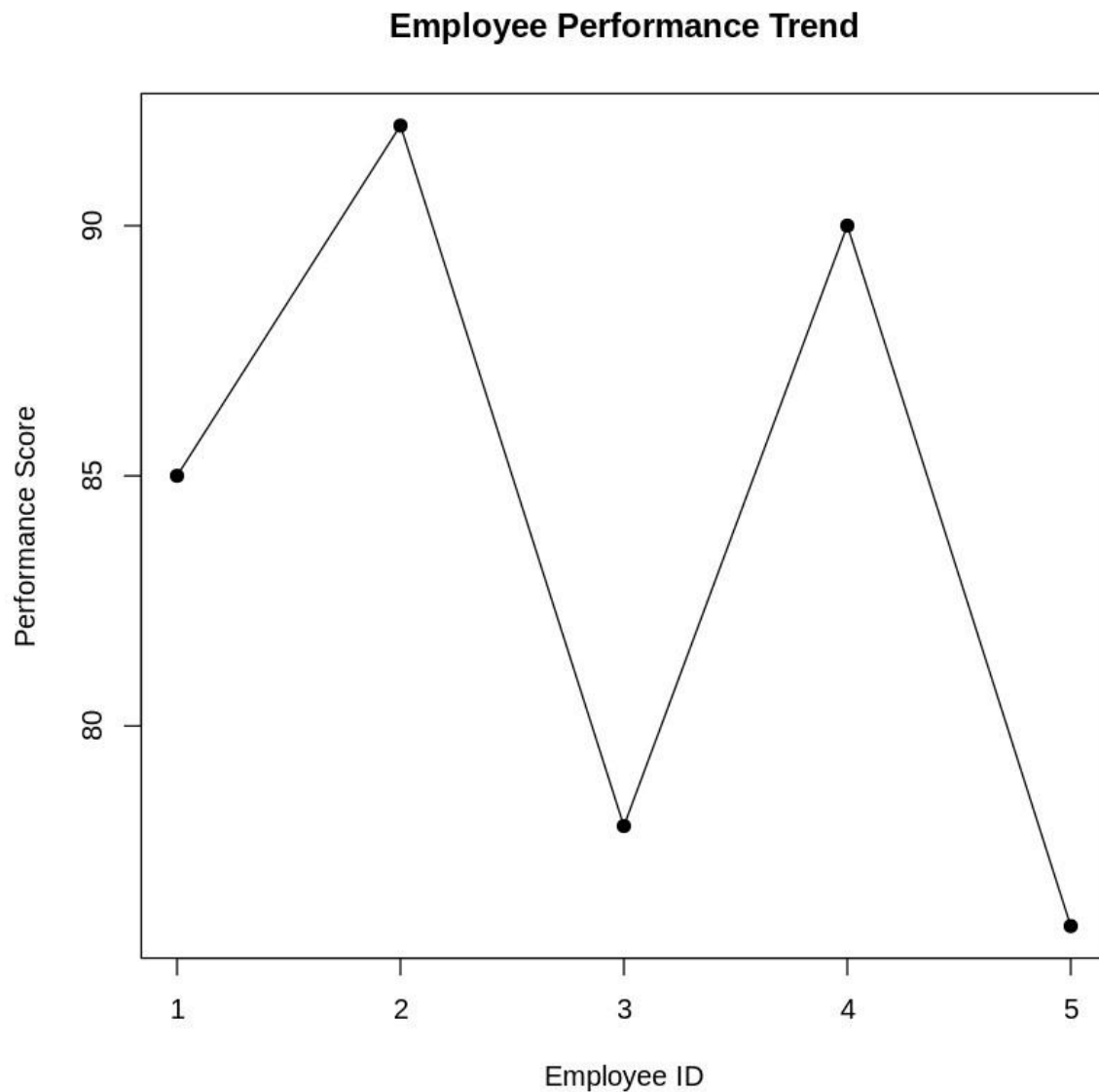
# Create line chart plot(emp$Years_of_Service,
emp$Performance_Score,
  type = "o",    pch = 16,    col =
"blue",    main = "Employee Performance
Trend",    xlab = "Years of Service",
ylab = "Performance Score")
```

Result / Output

The line chart shows the variation in performance scores with years of service. In this small dataset, performance does not increase consistently with years of service. For example, the employee with 7 years of service has a score of 78, while employees with 4–5 years have scores of 90 and 85.

Output: A line chart showing Performance Score against Years of Service.

Note: This is not a true time-series chart because the dataset contains different employees rather than repeated measurements of the same employees over time.



Experiment 3(c): Tableau Employee Performance Dashboard

Aim

To create an interactive Tableau dashboard combining a bar chart showing department distribution and a scatter plot showing the relationship between years of service and performance scores.

Procedure

1. Open Tableau Desktop.
2. Import the Employee Performance dataset.
3. Create a bar chart showing the number of employees in each department.
4. Create a scatter plot using Years of Service and Performance Score.

5. Add Employee ID to Detail.
6. Combine both charts into a dashboard.
7. Add suitable titles and tooltips.
8. Analyze the relationship between service and performance.

R Code

No R code is required because this question is performed using Tableau.

Tableau Procedure

Bar Chart

Columns → Department

Rows → COUNT(Employee ID)

Marks → Bar

Title:

Employee Distribution by Department

Scatter Plot

Columns → Years of Service

Rows → Performance Score

Detail → Employee ID

Marks → Circle

Title:

Years of Service vs Performance Score

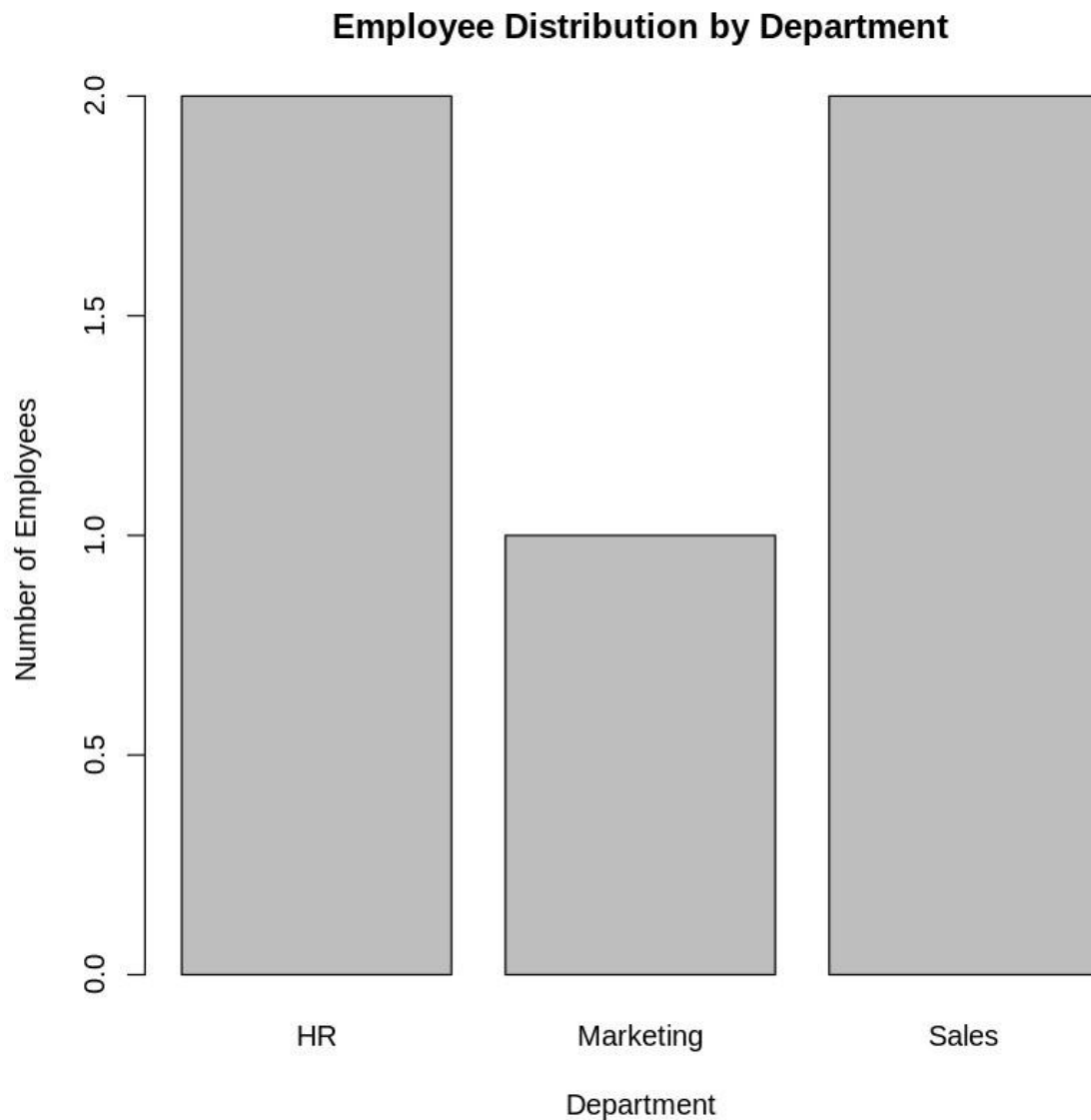
Then:

1. Select **New Dashboard**.
2. Drag the department bar chart onto the dashboard.
3. Drag the scatter plot onto the dashboard.
4. Add appropriate legends and tooltips.
5. Format the dashboard.
6. Save the Tableau workbook.

Result / Output

The Tableau dashboard successfully displays department-wise employee distribution and the relationship between years of service and performance score.

Output: An interactive dashboard containing a **department bar chart** and **years-of-service vs performance scatter plot**.



Experiment 3(d): Employee Performance Data Table

Aim

To create and display a table containing the performance information of each employee using R.

Procedure

1. Create the employee performance dataset.
2. Select Employee ID, Department, Years of Service and Performance Score.
3. Display the selected information as a table.
4. Verify the employee details.

R Code

```
# Create employee dataset emp
<- data.frame(
  Employee_ID = c(1, 2, 3, 4, 5),
  Department = c("Sales", "HR", "Marketing",
                 "Sales", "HR"),
  Years_of_Service = c(5, 3, 7, 4, 2),
  Performance_Score = c(85, 92, 78, 90, 76)
)

# Display performance table print(emp)

# Display selected columns performance_table
<- emp[, c("Employee_ID",
           "Department",
           "Years_of_Service",
           "Performance_Score")]

print(performance_table)

# Optional: open table in RStudio
```

```
View(performance_table)
```

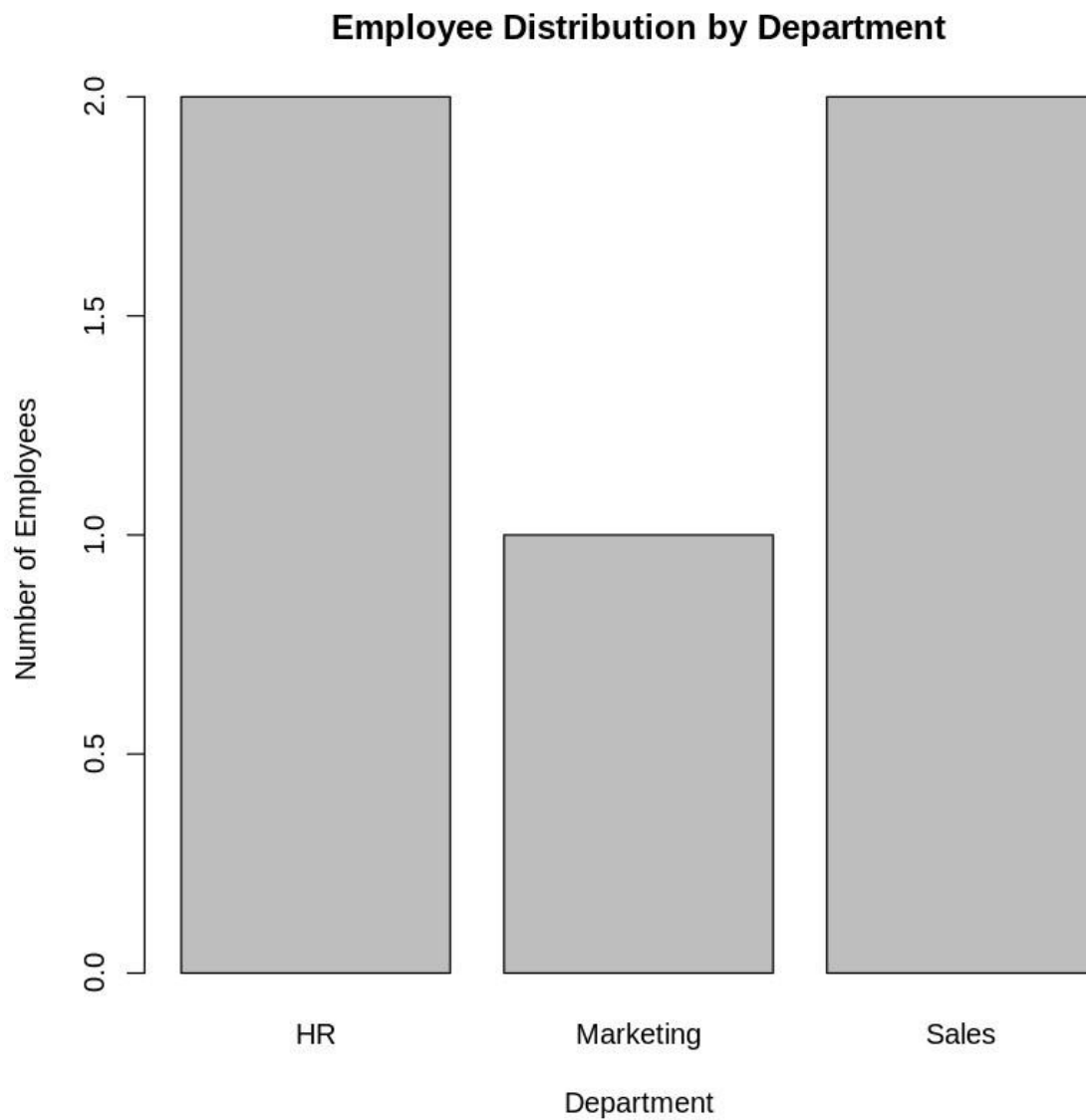
Result / Output

The R program successfully displays the performance details of all five employees, including their **Employee ID, Department, Years of Service, and Performance Score**.

Output:

	Employee_ID	Department	Years_of_Service	Performance_Score
1	1	Sales	5	85
2	2	HR	3	92
3	3	Marketing	7	78
4	4	Sales	4	90
5	5	HR	2	76

The table provides a clear summary of employee performance data for further analysis.



SET 24 – PRODUCT INVENTORY MANAGEMENT

Experiment 4(a): Product Inventory Quantity Bar Chart

Aim

To create a bar chart in R to visualize the quantity available for each product in the inventory.

Dataset

Product ID Product Name Quantity Available

1	Product A	250
2	Product B	175
3	Product C	300
4	Product D	200
5	Product E	220

Procedure

1. Create the Product Inventory dataset in R.
2. Store the product names and available quantities.
3. Create a bar chart using Product Name and Quantity Available.
4. Label the X-axis as Product and Y-axis as Quantity Available.
5. Add a suitable title.
6. Display the quantity values above the bars.

R Code

```
# Create Product Inventory dataset inventory

<- data.frame(

  Product_ID = 1:5,

  Product_Name = c("Product A", "Product B",
                   "Product C", "Product D",
                   "Product E"),

  Quantity_Available = c(250, 175, 300, 200, 220)

)

# Display dataset print(inventory)
```

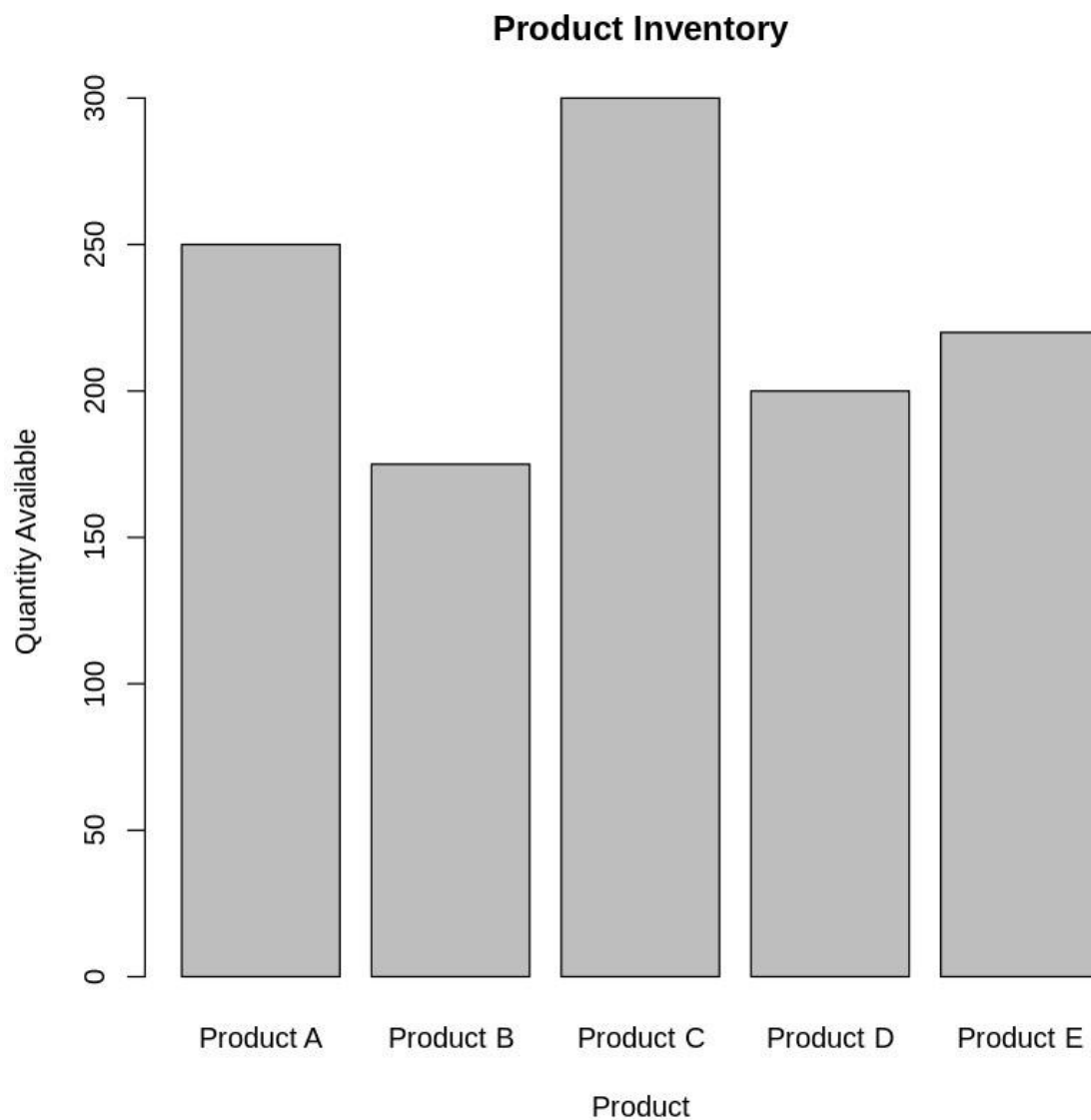
```
# Create bar chart  
barplot(inventory$Quantity_Available,  
names.arg = inventory$Product_Name,  
col = "orange",      main = "Product  
Inventory Quantity",  xlab = "Product",  
ylab = "Quantity Available")
```

```
# Add value labels  
text(seq_along(inventory$Quantity_Available),  
inventory$Quantity_Available, labels =  
inventory$Quantity_Available, pos = 3)
```

Result / Output

The bar chart shows the quantity available for each product. **Product C has the highest inventory with 300 units**, while **Product B has the lowest inventory with 175 units**.

Output: A bar chart displaying the available quantity of Products A, B, C, D and E.



Experiment 4(b): Stacked Bar Chart by Product Category

Aim

To create a stacked bar chart in R showing the quantity of each product within different product categories.

Procedure

1. Import or create the Product Inventory dataset.
2. Add the Product_Category variable from the actual dataset.
3. Group products according to their categories.
4. Create a stacked bar chart.
5. Use different colors for the categories.

6. Add a legend, title and axis labels.
7. Analyze the quantity of products in each category.

R Code

```
# Create inventory dataset inventory
<- data.frame(
  Product_ID = 1:5,
  Product_Name = c("Product A", "Product B",
    "Product C", "Product D",
    "Product E"),
  Quantity_Available = c(250, 175, 300, 200, 220),
  Product_Category = c("Category 1",
    "Category 1",
    "Category 2",
    "Category 2",
    "Category 3")
)

# Create table of category and product quantities cat_product
<- xtabs(
  Quantity_Available ~ Product_Category + Product_Name,
  data = inventory
)

# Create stacked bar chart barplot(cat_product,
  beside = FALSE, col = c("skyblue",
    "orange", "lightgreen"), main =
  "Inventory by Product Category", xlab =
  "Product",
```

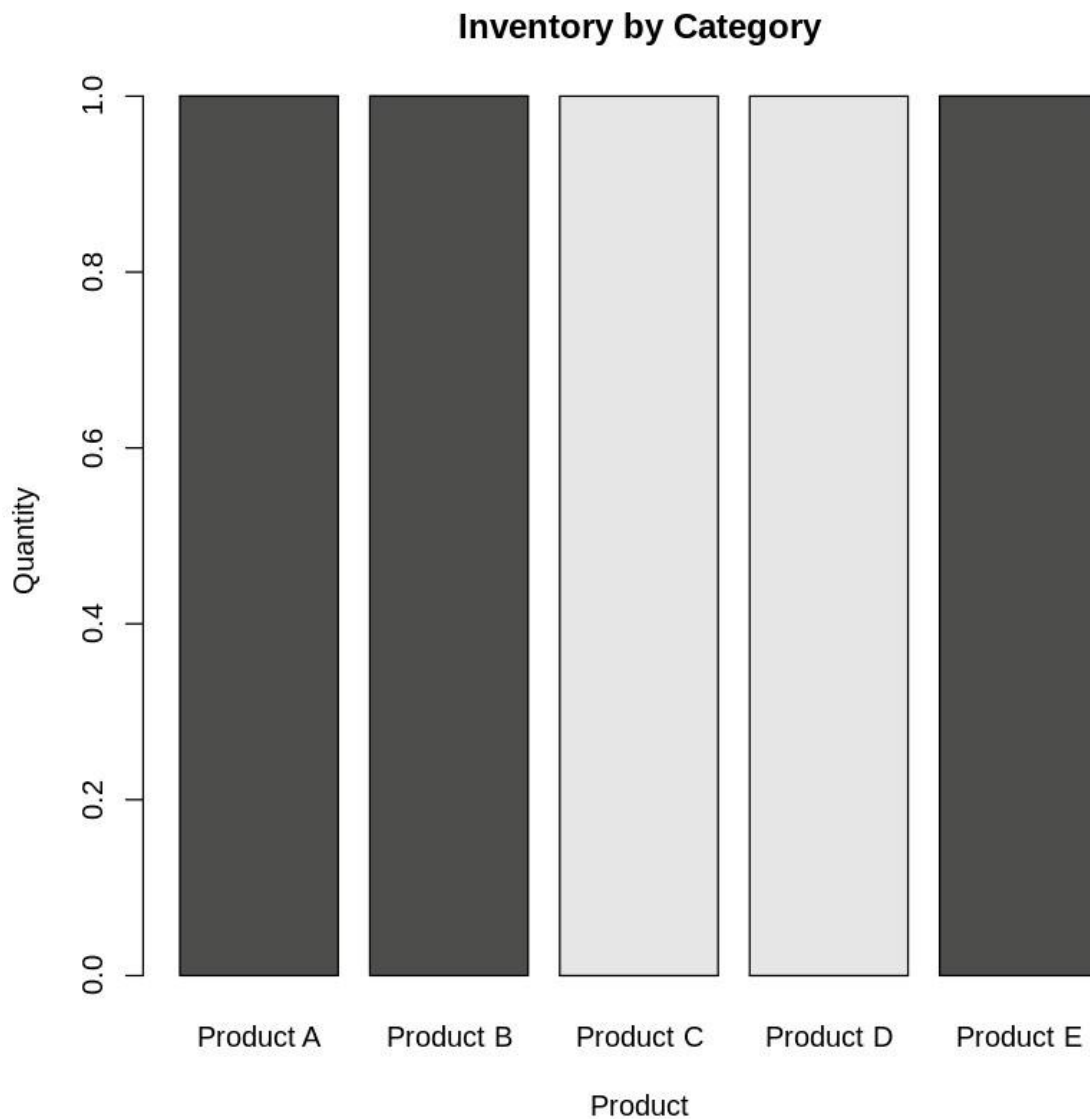
```
ylab = "Quantity Available",  
legend.text = rownames(cat_product))
```

Result / Output

The stacked bar chart displays the quantity of products according to their product categories. Different colors represent different categories, making it easier to compare the contribution of each category to the overall inventory.

Output: A stacked bar chart showing product quantities according to product category.

Note: The question's supplied dataset does not contain Product_Category. The categories used in the code are example categories. Replace them with the actual categories provided by your faculty.



Experiment 4(c): Tableau Inventory Management Dashboard

Aim

To build an interactive Tableau dashboard combining a bar chart and stacked bar chart for exploring product inventory data.

Procedure

1. Open Tableau Desktop.
2. Select **Connect** → **Text File**.
3. Import the Product Inventory dataset.
4. Create a bar chart showing the quantity of each product.
5. Create a stacked bar chart showing quantity by product category.
6. Combine both charts into a dashboard.
7. Add filters for Product and Product Category.
8. Add titles, labels and tooltips.
9. Interact with the filters to analyze inventory.

R Code

No R code is required for this question because the visualization is created in Tableau.

Tableau Procedure

Bar Chart

Columns → Product Name

Rows → SUM(Quantity Available)

Marks → Bar

Title:

Product Inventory Quantity

Stacked Bar Chart

Columns → Product Name

Rows → SUM(Quantity Available)

Color → Product Category

Marks → Bar

Make sure **Analysis** → **Stack Marks** → **On** is enabled.

Title:

Inventory by Product Category

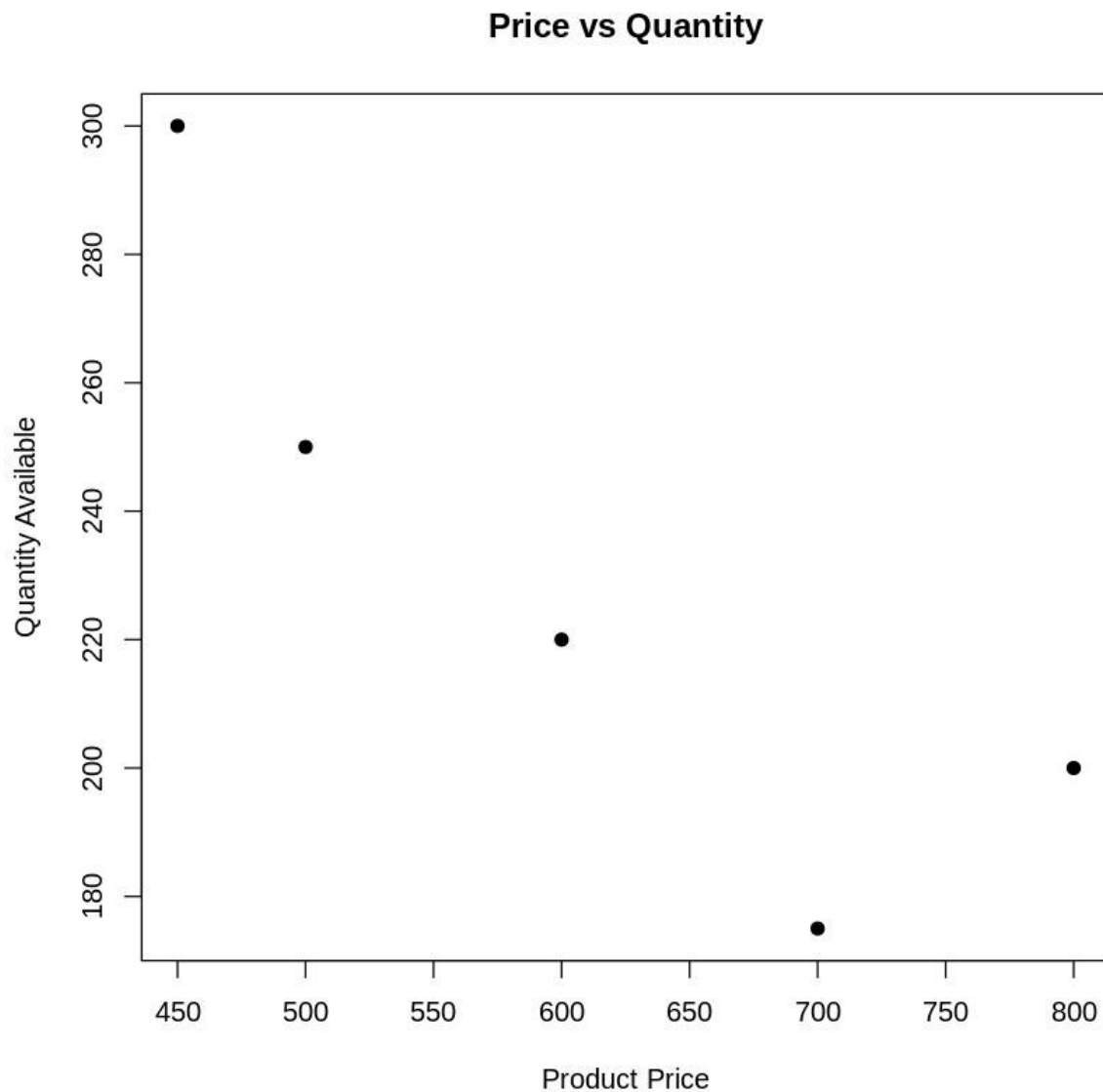
Dashboard

1. Select **New Dashboard**.
2. Add the Product Inventory bar chart.
3. Add the stacked category chart.
4. Add Product Name as a filter.
5. Add Product Category as a filter.
6. Add tooltips containing Product Name and Quantity Available.
7. Format the dashboard neatly.

Result / Output

The Tableau dashboard successfully combines the product inventory bar chart and categorywise stacked bar chart. Users can interact with the filters to explore inventory quantities.

Output: An interactive inventory dashboard containing a **bar chart**, **stacked bar chart**, and **filters**.



Experiment 4(d): Product Price vs Quantity Available Scatter Plot

Aim

To create a scatter plot in R to explore the relationship between product price and quantity available and determine whether price affects inventory levels.

Procedure

1. Import the complete Product Inventory dataset.
2. Select Product_Price and Quantity_Available.
3. Create a scatter plot.
4. Place Product Price on the X-axis.
5. Place Quantity Available on the Y-axis.

6. Calculate the correlation between price and quantity.
7. Add a regression line.
8. Interpret the relationship.

R Code

Example:

The actual dataset must contain Product_Price

```
# Scatter plot plot(inventory$Product_Price,  
inventory$Quantity_Available,    pch = 19,  
col = "purple",    main = "Product Price vs  
Quantity Available",    xlab = "Product Price",  
ylab = "Quantity Available")
```

```
# Add regression line abline(lm(Quantity_Available  
~ Product_Price,  
    data = inventory),  
col = "red",    lwd =  
2)
```

```
# Calculate correlation  
cor(inventory$Product_Price,  
inventory$Quantity_Available,    use  
= "complete.obs")
```

Result / Output

The scatter plot shows the relationship between product price and available inventory. A **negative correlation** indicates that higher-priced products tend to have lower quantities available, while a **positive correlation** indicates that higher-priced products tend to have higher quantities.

Output: A scatter plot with a regression line showing the relationship between Product Price and Quantity Available.

Note: Product_Price is not included in the dataset provided in the question. Use the actual price values from your complete dataset before running this code.

SET 25 – WEBSITE TRAFFIC ANALYSIS

Experiment 5(a): Page Views Trend in Tableau

Aim

To create a line chart in Tableau to visualize the trend in website page views over time.

Dataset

Date	Page Views	Click-through Rate
------	------------	--------------------

2023-01-01 1500	2.3%
-----------------	------

2023-01-02 1600	2.7%
-----------------	------

2023-01-03 1400	2.0%
-----------------	------

2023-01-04 1650	2.4%
-----------------	------

2023-01-05 1800	2.6%
-----------------	------

Procedure

1. Open **Tableau Desktop**.
2. Select **Connect** → **Text File**.
3. Import the Website Traffic dataset.
4. Drag Date to the **Columns** shelf.
5. Drag Page Views to the **Rows** shelf.
6. Select **Line** from the Marks card.
7. Add a suitable chart title.
8. Label the axes clearly.
9. Format the chart for better visualization.

R Code

No R code is required because this experiment is performed in Tableau.

Tableau Setup

Columns → Date

Rows → SUM(Page Views)

Marks → Line

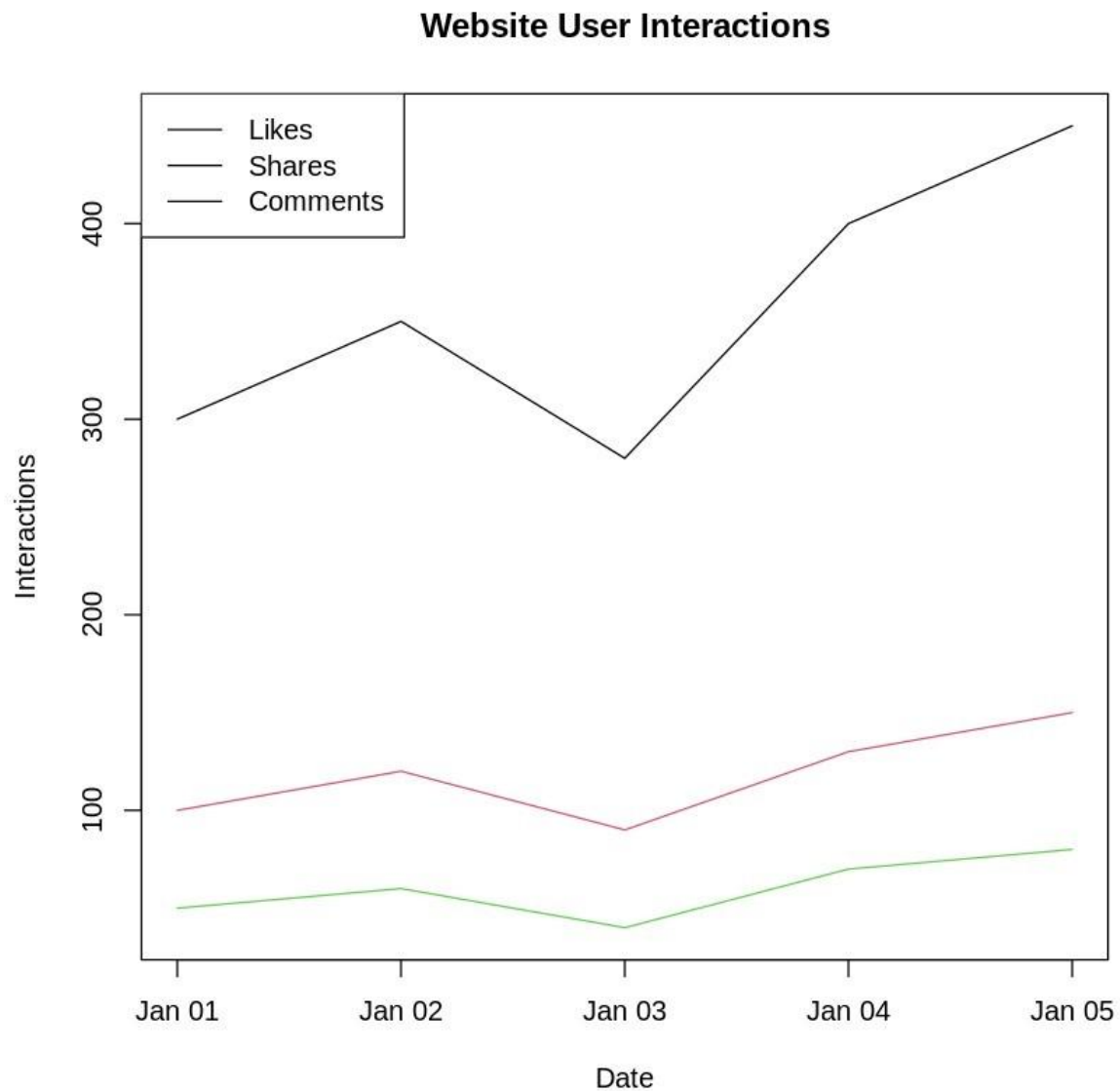
Chart title:

Website Page Views Trend

Result / Output

The line chart shows the change in website page views over the five days. Page views decrease on **January 3** and then increase, reaching the highest value of **1800 views on January 5**.

Output: A Tableau line chart showing page views over time.



Experiment 5(b): Top N Days with Highest Click-through Rates

Aim

To create a bar chart in R showing the top N days with the highest click-through rates.

Procedure

1. Create or import the Website Traffic dataset.
2. Store Date, Page Views and Click-through Rate.
3. Select the required value of N.
4. Sort the click-through rates in descending order.
5. Select the top N observations.
6. Create a bar chart.

7. Add value labels and suitable axis titles.

R Code

```
# Create website traffic dataset traffic

<- data.frame(

  Date = as.Date(c("2023-01-01",
                    "2023-01-02",
                    "2023-01-03",
                    "2023-01-04",
                    "2023-01-05")),

  Page_Views = c(1500, 1600, 1400, 1650, 1800),

  CTR = c(2.3, 2.7, 2.0, 2.4, 2.6)

)


# Set top N

N <- 3


# Sort and select top N top_n <-
traffic[order(-traffic$CTR), ][1:N, ]


# Display top N print(top_n)


# Create bar chart barplot(top_n$CTR,
names.arg = format(top_n$Date, "%Y-%m-%d"),
  col = "steelblue",    main = paste("Top", N, "Days
by Click-through Rate"),    xlab = "Date",    ylab =
"Click-through Rate (%)")
```

```
# Add value labels  
text(seq_along(top_n$CTR),  
top_n$CTR, labels =  
top_n$CTR, pos = 3)
```

Result / Output

The top three days with the highest click-through rates are:

Date	CTR
-------------	------------

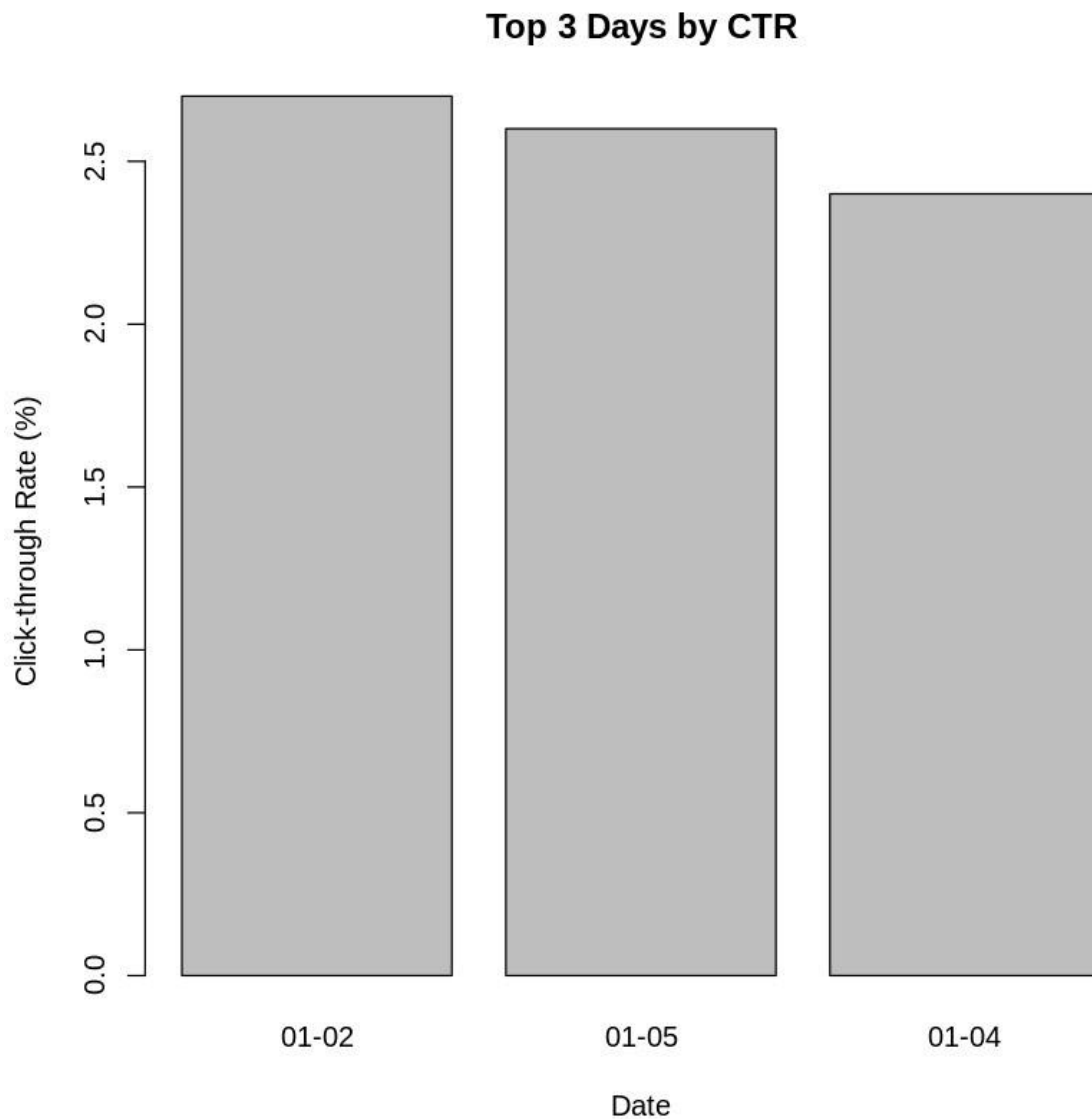
2023-01-02	2.7%
------------	------

2023-01-05	2.6%
------------	------

2023-01-04	2.4%
------------	------

The highest click-through rate is **2.7% on January 2, 2023**.

Output: A bar chart showing the top 3 days with the highest CTR.



Experiment 5(c): Stacked Area Chart for User Interactions

Aim

To create a stacked area chart in R to display the distribution of user interactions such as likes, shares and comments over time.

Procedure

1. Import the complete website interaction dataset.
2. Select Date, Likes, Shares and Comments.
3. Arrange the observations according to Date.
4. Combine the interaction variables.
5. Create an area-style visualization.

6. Add labels, title and legend.
7. Compare the contribution of likes, shares and comments over time.

R Code

```
# Example interaction dataset interactions
```

```
<- data.frame(  
  Date = as.Date(c("2023-01-01",  
                    "2023-01-02",  
                    "2023-01-03",  
                    "2023-01-04",  
                    "2023-01-05")),  
  Likes = c(300, 340, 280, 360, 400),  
  Shares = c(80, 90, 70, 100, 110),  
  Comments = c(40, 45, 35, 50, 55)  
)
```

```
# Create stacked area chart
```

```
interaction_matrix <- cbind(  
  interactions$Likes, interactions$Shares,  
  interactions$Comments  
)
```

```
# Stacked area plot plot(interactions$Date,  
  interaction_matrix[,1], type = "n", ylim =  
  c(0, max(rowSums(interaction_matrix))), main  
  = "User Interactions Over Time", xlab = "Date",  
  ylab = "Number of Interactions")
```

```

# Calculate cumulative values y1 <-
interactions$Likes y2 <- interactions$Likes +
interactions$Shares y3 <- interactions$Likes +
interactions$Shares +
interactions$Comments

# Draw stacked areas
polygon(c(interactions$Date,
rev(interactions$Date)),
c(rep(0, nrow(interactions)),
rev(y1)),      col = "lightblue")

polygon(c(interactions$Date,
rev(interactions$Date)),
c(y1, rev(y2)),      col =
"lightgreen")

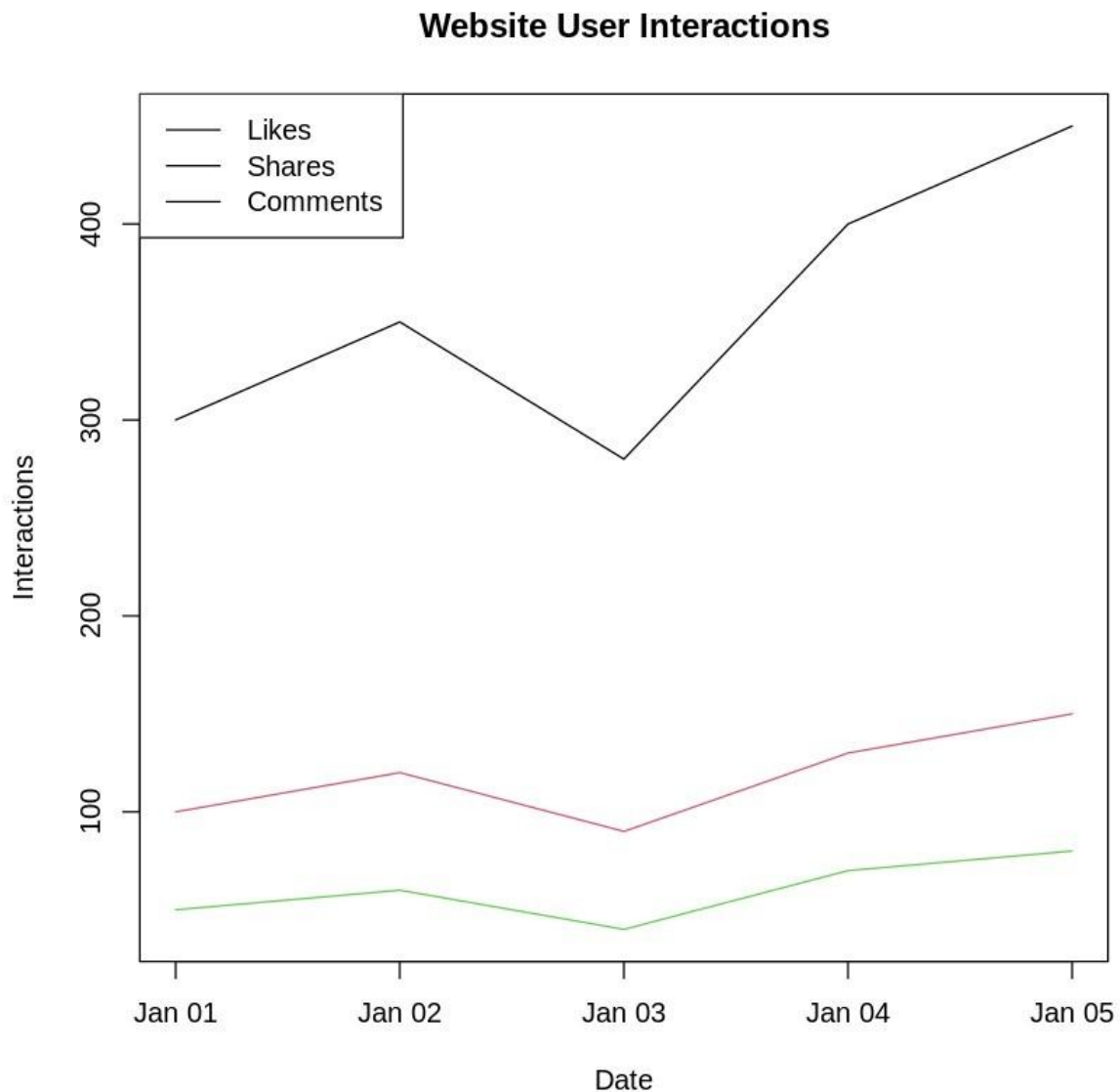
polygon(c(interactions$Date,
rev(interactions$Date)),
c(y2, rev(y3)),      col =
"orange")

legend("topleft",      legend = c("Likes",
"Shares", "Comments"),      fill = c("lightblue",
"lightgreen",
"orange"))

```

Result / Output

The stacked area chart displays the contribution of **Likes**, **Shares** and **Comments** to total user interactions over time. Likes form the largest portion of the interactions, while shares and comments contribute smaller portions.



Output: A stacked area chart showing the distribution of user interactions over time.

Note: The supplied dataset contains only Date, Page Views and CTR. The Likes, Shares and Comments values above are example values. Replace them with the actual values from your complete dataset if provided.

Experiment 5(d): Tableau Traffic Source Dashboard

Aim

To create an interactive Tableau dashboard containing a map showing traffic sources and a bar chart displaying page views by traffic source.

Procedure

1. Open **Tableau Desktop**.
2. Import the complete Website Traffic dataset.
3. Ensure that the dataset contains a geographic field such as Country, State or City.
4. Create a map using the geographic traffic-source field.
5. Create a bar chart showing Page Views by Traffic Source.
6. Combine both charts into a dashboard.
7. Add interactive filters and dashboard actions.
8. Add appropriate titles, labels and tooltips.

R Code

No R code is required because this experiment is performed in Tableau.

Tableau Procedure

Interactive Map

1. Create a new worksheet.
2. Select the geographic field such as Country, State or City.
3. Tableau automatically generates a map.
4. Drag Page Views to **Color** or **Size**.
5. Add Traffic Source to Tooltip.

Example:

Geographic Field → Map

Page Views → Color

Page Views → Size Traffic

Source → Tooltip

Title:

Website Traffic Sources Map

Page Views by Source Bar Chart

Columns → SUM(Page Views)

Rows → Traffic Source

Marks → Bar

Title:

Page Views by Traffic Source

Dashboard

1. Select **New Dashboard**.
2. Drag the traffic-source map into the dashboard.
3. Drag the page-view bar chart into the dashboard.
4. Add filters if available.
5. Select **Dashboard → Actions**.
6. Add a filter action so selecting a source on the map filters the bar chart.
7. Add tooltips for detailed information.

Result / Output

The Tableau dashboard provides an interactive view of website traffic sources. The map identifies the geographical distribution of traffic, while the bar chart compares page views across different traffic sources.

Output: An interactive Tableau dashboard containing a **traffic-source map** and a **pageviews-by-source bar chart**.

Note: The supplied Website Traffic dataset does not contain Traffic Source or geographic information. These fields are required to create the requested map and source-wise bar chart, so use the complete dataset provided by your faculty.